SAiDL Summer Induction Assignment 2026

Reinforcement Learning: Transformer Actors in TD3 under Full and Partial Observability

Aarush Gupta
2023B3A70839G

Contents

1 Introduction

Standard reinforcement learning algorithms like TD3 use memoryless MLP policies. At each timestep the agent sees only the current observation and must produce an action. This is perfectly adequate when the environment is fully observable and Markovian — the current state contains everything needed to act optimally.

Real environments often break this assumption. Sensors are noisy. Some state components are simply not observable. Rewards may arrive long after the actions that caused them. In all of these cases, the current observation alone is not enough, and an agent that can remember and reason over its history should, in theory, do better.

Transformers are a natural candidate for this role. A causal Transformer operating on a sliding window of past observation-action pairs can in principle use self-attention to identify which past timesteps are most relevant to the current decision. This is a learned form of working memory, and it is exactly what a POMDP agent needs.

This report asks a simple question: does replacing the MLP actor in TD3 with a causal Transformer actually help, and if so, under what conditions? We train both variants on `Hopper-v5` under four settings — full observability, hidden velocities, observation noise, and delayed rewards — and compare their performance.

The short answer, which we discuss honestly throughout, is that the Transformer actor struggled to learn in all of our experiments. Whether this reflects a fundamental limitation of Transformer policies in online RL, or simply the cost of training them under our compute constraints, is the central question this report tries to answer.

2 Background

2.1 TD3

Twin Delayed Deep Deterministic Policy Gradient (TD3) [1] is an off-policy actor-critic algorithm for continuous action spaces. It addresses overestimation bias in DDPG by maintaining two critic networks and taking the minimum of their Q-value estimates when computing the target. The actor is updated less frequently than the critics (policy delay) to give the critic estimates time to stabilise before the actor follows them. Target networks for both actor and critics are maintained with Polyak averaging.

2.2 Transformer Actor

In our Transformer-TD3 variant, the MLP actor is replaced by a causal Transformer that receives a sliding window of the last L observation-action pairs as a sequence. Causal self-attention is applied across this window, and the output at the final (most recent) position is used to produce the action. The Transformer thus has access to the agent's recent history, rather than only the current observation.

The intuition is that attention weights can learn to focus on the past timesteps most relevant to the current decision. Under full observability, this should be redundant — the current observation already contains the full state. Under partial observability, it should help, because past observations carry information that the current one does not.

2.3 Partial Observability Settings

We apply three independent POMDP perturbations to `Hopper-v5`:

Hidden velocities. The velocity components of the observation are removed. Hopper's observation space has 11 dimensions (plus the x-position); velocity information occupies the last few dimensions. Without these, the agent must infer velocity from the change in position across consecutive timesteps — which is exactly the kind of temporal integration a memory-enabled policy could do.

Observation noise. Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$ is added to every observation at every step. We use $\sigma = 0.3$. The agent cannot distinguish between true state changes and noise fluctuations from a single observation, but averaging over a history of observations could reduce the effective noise variance.

Delayed rewards. Instead of receiving a reward at every step, the agent receives the accumulated sum of rewards every $K = 10$ steps, and zero otherwise. This breaks the tight temporal credit assignment that standard TD3 relies on, and requires the agent to attribute reward to actions taken several steps earlier.

We also ran a combined POMDP setting (all three perturbations simultaneously) for the MLP baseline, which is discussed in Section 4.5.

3 Implementation

3.1 Architecture and Configuration

The implementation follows the preferred configuration specified in the assignment. The Transformer actor uses 2 layers, 4 attention heads, and an embedding dimension of 128. Pre-layer normalisation (Pre-LN) is used for training stability. Running mean/std normalisation is applied to observations. The context window is fixed at $L = 8$ for all POMDP experiments. We also ran $L = 4$ for the full observability setting as part of a partial sweep (see Section 4.1).

TD3 hyperparameters follow the assignment specification: learning rate 3×10^{-4}, replay buffer size 10^6, batch size 256, Polyak coefficient $\tau = 0.005$, and policy delay 2. Evaluations are run every 5,000 steps over 10 deterministic rollout episodes. All runs use a base seed of 42.

The full configuration is managed through a single `TD3config` dataclass with boolean flags for each POMDP variant, making it straightforward to run controlled comparisons where only one condition changes at a time.

3.2 A Note on Training Speed

The Transformer actor is substantially more expensive to train than the MLP. Our initial setup on Windows with `n_envs = 3` (three parallel environments) produced only 8–10 steps per second on an RTX 5090 — far too slow to reach 1,000,000 training steps in reasonable time.

We switched to Linux to enable `torch.compile`, which provides significant speedups through graph-level optimisation of the forward pass. We also increased `n_envs = 32`

to collect more environment transitions per wall-clock second, which brings the effective throughput to roughly 60–80 steps per second for the Transformer runs and over 200 steps per second for the MLP runs (see Table 6).

Despite these optimisations, some Transformer runs did not reach the full 1,000,000 training steps. The L8 full-observability run crashed at around 20,000 steps due to an unrecovered error. The delayed reward and observation noise Transformer runs were manually terminated at approximately 587,000 and 654,000 steps respectively due to time constraints. The hidden velocity Transformer run reached 944,000 steps before being stopped. All MLP runs completed the full 1,000,000 steps.

We report results as-is and note where step count differences may affect comparison.

4 Experiments and Results

For each setting we report the best smoothed evaluation return (smoothed over a 5-step window to reduce noise), the evaluation return at the final logged step, and the standard deviation of evaluation returns at the best checkpoint. All evaluation returns are averaged over 10 deterministic rollout episodes.

4.1 Full Observability

Table 1: Full observability results on `Hopper-v5`. TF = Transformer actor. The L8 Transformer crashed at 20K steps; the L4 run was still in progress at 717K steps.

Agent	Steps	Best Eval Return	± Std	Final Eval Return
MLP-TD3	1,000,000	1121.6	275.3	1313.8
TF-TD3 (L=4)	716,900	23.4	18.3	14.3
TF-TD3 (L=8)	19,992	21.0	40.8	18.7

The MLP baseline achieves a best evaluation return of 1121.6 and is still improving at the end of training, reaching 1313.8 at the final checkpoint. This is a healthy result for Hopper — the agent has clearly learned to walk.

The Transformer runs tell a very different story. The L=8 run crashed before it could meaningfully learn anything (20K steps is well within the initial random-policy phase where the replay buffer is still sparse). The L=4 run was still running at the time of writing, at 717K steps, but with a best return of 23.4 — essentially the performance of a random policy. The L=4 run shows no meaningful upward trend in evaluation returns across its logged history.

This is a notable result. Under full observability, the Transformer has no theoretical advantage, but we would still expect it to eventually learn to walk given enough steps. The fact that it has not by 700K steps suggests that the Transformer is significantly harder to train in this online RL setting — the credit assignment problem is the same, but the additional capacity and the sequential nature of the Transformer's input appear to make optimisation substantially harder.

4.2 Hidden Velocities

Table 2: Hidden velocity results on `Hopper-v5`. Velocity components removed from observations. TF run killed at 945K steps.

Agent	Steps	Best Eval Return	± Std	Final Eval Return
MLP-TD3	1,000,000	38.0	1.1	38.3
TF-TD3 (L=8)	944,679	25.8	13.1	21.2

This setting is the most theoretically interesting. Removing velocity information from the observation makes the environment genuinely non-Markovian: knowing only the current joint angles is not enough to determine an optimal action, because the same configuration of joints can correspond to very different velocity states (the hopper could be moving forward quickly, slowly, or even falling).

The MLP result here is striking in a different way. Without velocity information, the MLP baseline collapses almost completely, achieving only 38.0 — barely above a random policy. This makes sense: an MLP with no memory simply cannot recover velocity information from a single observation.

But the Transformer does not do better. With access to a window of $L = 8$ past observations, it could in principle compute a finite-difference approximation of velocity from consecutive position readings, but its best return of 25.8 is actually lower than the MLP's 38.0. The Transformer's evaluation returns are also more variable (std 13.1 vs 1.1 for the MLP), suggesting it has not found a stable policy at all.

Why does the Transformer not take advantage of its context window here? We think the most likely explanation is again the difficulty of online optimisation. The Transformer needs to jointly learn a useful representation from the observation history *and* a good policy on top of it, all via sparse reward signals from an environment where neither the MLP nor the Transformer has found a working policy. The MLP, being simpler, at least converges to a consistent low-return policy. The Transformer's larger parameter space and more complex input structure makes even that harder.

4.3 Observation Noise

Table 3: Observation noise results on `Hopper-v5` ($\sigma = 0.3$). TF run still in progress at 654K steps.

Agent	Steps	Best Eval Return	± Std	Final Eval Return
MLP-TD3	1,000,000	711.9	77.1	465.4
TF-TD3 (L=8)	654,196	22.5	40.3	23.1

The MLP degrades gracefully under noise. At $\sigma = 0.3$, it achieves a best return of 711.9 — down from 1121.6 under full observability, but still a genuinely functional walking policy. The noise makes the policy less consistent (the final return of 465.4 is notably lower than the best, suggesting some instability), but the agent has clearly learned something useful.

This robustness is perhaps unsurprising: TD3's target policy smoothing already adds noise to actions during training, so the MLP is partially implicitly trained to handle noisy inputs.

The Transformer again does not learn, with a best return of 22.5 at 654K steps. Note that the eval std of 40.3 for the Transformer is nearly as large as its mean return — the policy is essentially random and highly variable.

4.4 Delayed Rewards

Table 4: Delayed reward results on `Hopper-v5` ($K = 10$). TF run killed at 587K steps.

Agent	Steps	Best Eval Return	± Std	Final Eval Return
MLP-TD3	1,000,000	3002.6	385.0	1640.8
TF-TD3 (L=8)	587,091	25.2	21.2	9.5

The MLP's performance under delayed rewards is the most surprising result in the study. With $K = 10$, the agent accumulates 10 timesteps of reward before receiving any signal, meaning the reward at each non-zero step is roughly 10 times the per-step reward in the undiscounted setting. This inflates the raw return numbers — the best return of 3002.6 should be compared against the undiscounted baseline of around 1121.6, and accounting for the summation it is roughly consistent.

More interestingly, the MLP does not collapse under this delay. TD3's off-policy nature likely helps here: by sampling from a replay buffer of diverse past transitions, the critic can still receive useful gradient signal even when the in-environment reward is sparse. The Bellman backup will propagate the accumulated reward backward through the stored transitions over many updates.

The Transformer's failure in this setting (best return 25.2 at 587K steps) is harder to explain as a memory advantage story, since the delay is only 10 steps — well within the context window of $L = 8$ (and arguably not requiring memory at all if the critic handles credit assignment correctly). This again points to the core problem being training stability, not the theoretical suitability of the architecture.

4.5 Combined POMDP (MLP Baseline Only)

Table 5: Combined POMDP setting: hidden velocities + observation noise ($\sigma = 0.3$) + delayed rewards ($K = 10$), MLP actor only.

Agent	Steps	Best Eval Return	± Std	Final Eval Return
MLP-TD3	1,000,000	316.1	9.9	315.3

We ran the MLP baseline under all three POMDP perturbations simultaneously as an additional experiment. The combined setting is substantially harder — the agent loses velocity information, has noisy observations, and receives delayed rewards all at once. The

result of 316.1 is lower than any individual POMDP condition except hidden velocities, but the policy is remarkably stable (std of only 9.9 and almost no gap between best and final return). The MLP appears to have found some consistent low-return behaviour — likely a shuffling gait rather than proper walking — that is robust to all three perturbations.

We did not run the Transformer-TD3 in the combined setting due to compute constraints.

4.6 Throughput Summary

Table 6: Median training throughput (steps per second) across all runs. MLP runs used `n_envs = 32` without `torch.compile`; Transformer runs used `n_envs = 32` with `torch.compile` on Linux.

Condition	Agent	Median Steps/sec
Full Obs	MLP	145.0
Full Obs (L=8)	Transformer	21.7
Full Obs (L=4)	Transformer	25.0
Hidden Vel	MLP	205.6
Hidden Vel	Transformer	78.7
Obs Noise	MLP	211.4
Obs Noise	Transformer	82.2
Delayed Rewards	MLP	213.5
Delayed Rewards	Transformer	78.8
Combined POMDP	MLP	204.4

The throughput gap between MLP and Transformer is large. The MLP runs with `n_envs = 32` achieve between 145 and 213 steps per second depending on the condition. The Transformer runs achieve 21–82 steps per second. The large range for Transformer runs reflects the difference between the early full-observability crash (21 steps/sec) and the POMDP runs that benefited from the Linux `torch.compile` setup (78–82 steps/sec). Even at 82 steps/sec, the Transformer is roughly 2.5× slower than the MLP.

5 Discussion

5.1 Why Did the Transformer Not Learn?

The Transformer actor never exceeded a return of 26 across all conditions. This is the central result and it needs an honest explanation.

The most likely reason is how we store and sample experience. Our replay buffer stores individual transitions — a single (observation, action, reward, next observation) tuple at a time. When we train the Transformer, we reconstruct a context window of L past transitions by stitching together stored tuples. These tuples may come from completely different parts of different episodes. The Transformer sees a sequence that never actually happened, which makes learning a coherent history-based policy very hard.

A second reason is that early training is especially painful for Transformers. At the start, the replay buffer is full of random actions. An MLP just needs to learn which action is

better given the current observation — a relatively simple mapping. The Transformer needs to learn which parts of a chaotic history of random actions are worth attending to. That is a much harder problem to start from.

Finally, even at 945K steps, there was no sign of improvement in the Transformer runs. The gap — MLP at 1000+ return versus Transformer near 25 — is too large to explain by just needing more steps.

5.2 How Hopper Shaped Every Result

This is something we should be upfront about: **all of our results are specific to Hopper-v5**, and Hopper is a somewhat unusual environment. Understanding how it is different helps make sense of what we saw.

Hopper is a continuous control task where a one-legged robot has to learn to hop forward without falling. The reward is dense — the agent gets a signal at every step based on forward velocity and staying upright. This makes it relatively easy for an MLP to learn, because feedback arrives constantly.

Why the MLP collapsed under hidden velocities. Hopper's dynamics are very sensitive to velocity. The same joint angle can mean the robot is mid-hop (fine) or about to fall (bad), and the difference is entirely in velocity. Removing velocity from the observation essentially removes the most important part of the state for Hopper specifically. In a different environment — say, one where the agent moves slowly and position alone is informative — hiding velocity might matter much less.

Why delayed rewards did not hurt the MLP much. Hopper's reward is dense and consistent. Accumulating 10 steps of reward just gives the agent a bigger number at every 10th step, but the *information content* is the same. The MLP has seen enough of these accumulated signals that the critic can still learn. In a sparse-reward environment (like a maze where reward only arrives at the goal), $K = 10$ delay would likely be far more damaging.

Why observation noise ($\sigma = 0.3$) was survivable. Hopper has relatively smooth dynamics. The state does not change dramatically between timesteps, so adding Gaussian noise with $\sigma = 0.3$ corrupts each observation but the underlying pattern is still recoverable. In a fast-moving or contact-rich environment (like a robot hand manipulating an object), the same noise level might completely destroy the signal.

What a different environment might have shown. A Transformer actor might look much better on an environment where memory genuinely helps and the dynamics are slower. For example: a maze navigation task (sparse reward, requires remembering which paths were explored), or a flickering observation task where the agent must integrate noisy signals over many steps. Hopper punishes the Transformer for its slow training without giving it any scenario where its memory provides a clear advantage that a well-tuned MLP cannot partially compensate for.

To draw reliable conclusions about whether Transformers help in POMDPs, the same experiments should be repeated on at least two or three different environments across multiple seeds. Our results are a data point about Hopper, not a general verdict.

5.3 What the MLP Results Tell Us

Even without a working Transformer comparison, the MLP results are worth reading.

The MLP survives noise and delayed rewards reasonably well. This tells us that TD3's off-policy training — where the critic learns from a diverse buffer of past transitions — is itself a form of robustness. The critic can receive delayed reward signals and still propagate them backward through stored experience.

The hidden velocity collapse (return of 38) is the clearest result in the study. It confirms that the information removed by hiding velocities is not recoverable from a single Hopper observation. This is the setting where we most wanted the Transformer to step up — and where the training difficulty prevented us from finding out if it would.

5.4 What Would Help

A few concrete changes that could make the Transformer work:

- **Store full trajectories.** If the replay buffer stores full episodes rather than individual transitions, the context windows fed to the Transformer would be real sequences from a single episode. This is how Recurrent Replay Distributed DPG [3] handles recurrent policies.
- **Smaller Transformer.** A single-layer, two-head Transformer would be faster to train and might find a working policy before the compute budget runs out.
- **Multiple environments.** Testing on environments where memory is more clearly beneficial (sparse reward, slow dynamics, longer dependencies) would give the Transformer a fairer chance to show its value.
- **Multiple seeds.** Three seeds per condition is the minimum for RL results to be meaningful. A single seed can easily be an outlier.

6 Limitations

Single environment, single seed. Every result in this report comes from one run on one environment. Hopper-v5 is a specific task with specific properties (dense reward, velocity-sensitive dynamics) that shaped our results in ways we described in Section 5.2. None of our conclusions generalise beyond Hopper without further experiments.

Single seed. RL training is sensitive to random seed — the same algorithm with a different seed can give very different learning curves. The assignment asks for 3 seeds. We ran 1. Any of our numbers could look different with a different seed.

Incomplete Transformer sweep. We only ran $L = 4$ and $L = 8$. The $L = 8$ full-observability run crashed at 20K steps. We never tested $L = 16$ or $L = 32$.

Truncated Transformer runs. Three Transformer runs did not reach 1,000,000 steps (range: 587K–945K). Direct comparison to MLP runs that all finished at 1M steps is not clean.

No attention analysis. The logger has the code for attention weight and entropy logging (`log_attention`), but we did not run the rollout analysis. This is a gap.

No RLHF, one noise level. RLHF was not implemented. We only ran $\sigma = 0.3$ for observation noise, not $\sigma = 0.1$.

7 Conclusion

We implemented TD3 with MLP and Transformer actors and tested them on `Hopper-v5` under full observability and three POMDP conditions.

The MLP learned to walk under full observability (best return ∼1122) and held up reasonably well under observation noise (∼712) and delayed rewards (∼3003, inflated by the $K = 10$ sum). It collapsed under hidden velocities (∼38), because Hopper specifically depends on velocity information in a way that a memoryless policy cannot work around.

The Transformer actor did not learn in any condition, with a best return under 26 across all runs. We think the main reasons are: context windows built from fragmented replay buffer transitions, the difficulty of learning from random-action histories at the start of training, and the raw compute cost of training a Transformer in an online loop.

The interesting question — does a memory-enabled policy actually do better when memory matters? — remains open. The hidden velocity result is exactly the scenario where the answer should be yes. But we could not get the Transformer to train well enough to find out. That is an honest summary of where we ended up, and it points directly to what should be done next: smaller Transformer, trajectory-based replay, and experiments on environments that are kinder to slow-training policies.

References

[1] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[2] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch. Decision Transformer: Reinforcement learning via sequence modeling. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[3] S. Kapturowski, G. Ostrovski, J. Quan, R. Munos, and W. Dabney. Recurrent experience replay in distributed reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2019.

[4] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[5] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

SAiDL Summer Induction Assignment 2026

Core ML: Long-Context Sequence Modelling

Attention Variants, Positional Encodings, and Conv–Attention Hybrids

Aarush Gupta
2023B3A70839G

Contents

1 Introduction

Standard self-attention scales quadratically with sequence length in both compute and memory. At a context window of $T = 1024$, the attention matrix alone is $T \times T = 1{,}048{,}576$ entries per head per layer. This is the central bottleneck that motivates a large body of work on efficient attention.

This report documents a systematic empirical study on the WikiText-2 dataset using a decoder-only Transformer. We ask three questions. First, how do alternative attention mechanisms — local, sparse block, and multi-query — compare against standard attention in terms of perplexity, memory, and latency? Second, how does the choice of positional encoding affect in-distribution performance, and does it affect the model's ability to generalise to longer sequences than it was trained on? Third, does adding a 1D convolutional component alongside attention improve results?

The study is not exhaustive. Compute constraints limited us to context lengths of 512 and 1024, and the small size of WikiText-2 required aggressive regularisation throughout. We document these constraints honestly where they affect interpretation.

2 Experimental Setup

2.1 Dataset

All experiments use WikiText-2, a standard language modelling benchmark built from verified Wikipedia articles. The dataset is small by modern standards — roughly 2M training tokens — which turned out to be a meaningful constraint. Early runs with $d_{\text{model}} = 256$ overfit badly; training loss continued to fall while validation loss diverged. We responded by reducing d_{model} to 64 and raising dropout to 0.3. All reported results use this configuration.

2.2 Model Configuration

The base model is a decoder-only Transformer following the architecture of Vaswani et al. [1], with the following fixed hyperparameters across all experiments:

- Layers: 2
- Attention heads: 4
- Embedding dimension d_{model}: 64
- Feed-forward dimension d_{ff}: 256 ($4 \times d_{\text{model}}$)
- Dropout: 0.3
- Training iterations: 30,000
- Batch size: 32
- Tokeniser: GPT-2 BPE (via `tiktoken`)

The architecture is modular: attention type, positional encoding, and convolutional components can each be swapped independently via a single configuration dataclass. This allowed us to run controlled comparisons where only one component changes at a time.

2.3 Training and Evaluation

Models were trained using a cosine learning rate schedule with warmup. Validation loss was computed on a held-out split using sequential, non-overlapping batches to ensure deterministic and comparable results across runs. Perplexity is reported as exp(val loss).

All experiments were tracked using Weights & Biases. Inference metrics (latency, throughput) were measured separately after training on a fixed hardware setup using 20 repeated generation

sequences of 128 tokens each, with 3 warmup passes before timing.

2.4 Hardware and a Limitation

T=512 training runs were conducted on an NVIDIA A100. T=1024 runs were conducted on an NVIDIA L40S due to compute availability. This means latency and memory numbers *are not directly comparable* across context lengths — the T=1024 numbers reflect both the longer context and a different GPU. Perplexity comparisons across context lengths are still valid since they measure model quality, not hardware performance.

3 Task 1: Attention Mechanisms

3.1 Variants

We implemented four attention mechanisms. The following is a brief description of each.

Standard self-attention computes a full $T \times T$ attention matrix. Every token can attend to every other token in a single step, which gives the model maximum flexibility to learn arbitrary dependency patterns. The cost is $\mathcal{O}(T^2)$ in both compute and memory.

Sliding window (local) attention restricts each token to attend only within a fixed window of w neighbours. The attention matrix becomes a band matrix. Complexity drops to $\mathcal{O}(T \cdot w)$. Most syntactic dependencies in natural language are local, so this works well for a large class of tasks, but long-range dependencies must propagate through many layers. We used $w = 128$.

Sparse block attention divides the sequence into blocks of size b and restricts attention to within-block. Tokens at block boundaries cannot directly attend to their immediate neighbours across the boundary, which is an artificial discontinuity. We used $b = 64$.

Multi-query attention (MQA) keeps multiple independent query heads but shares a single key and value projection across all heads. This does not change the $\mathcal{O}(T^2)$ training complexity, but reduces the KV cache at inference by a factor of H (the number of heads), which is the dominant memory cost during long-context generation.

3.2 Results

Table 1: Task 1 — Attention variant comparison on WikiText-2 (2 layers, 4 heads, $d_{\text{model}} = 64$, dropout=0.3, 30k iterations). T=512 runs on NVIDIA A100; T=1024 runs on NVIDIA L40S. Latency and memory figures are **not** directly comparable across context lengths due to hardware differences.

Attention	Context	Val PPL ↓	Peak GPU (MB)	Throughput (tok/s) ↑	Latency (ms/seq) ↓
Standard	512	106.24	10,503	192.61	664.54
Local	512	100.35	10,341	93.18	1,373.62
MQA	512	108.30	10,472	217.45	588.64
Standard	1024	106.06	21,726	179.24	714.13
Local	1024	97.94	20,622	67.74	1,889.54
MQA	1024	106.78	21,663	209.55	610.82
Sparse	1024	107.83	20,241	186.00	688.16

Note: Sparse attention was only run at T=1024. A T=512 sparse run was not completed due to time constraints.

3.3 Analysis

The most notable result is that **local attention consistently achieves the lowest perplexity** at both context lengths (100.35 at T=512, 97.94 at T=1024). This is somewhat counterintuitive at first glance — local attention has strictly less expressive power than standard attention — but it makes sense given the dataset. WikiText-2 consists of Wikipedia articles where local context dominates. Most of what a language model needs to predict the next token is in the immediately surrounding text. Local attention's inductive bias toward locality turns out to be well-matched to this data.

Standard and MQA attention produce nearly identical perplexity at both context lengths (within 2 PPL of each other). This is consistent with theory: MQA reduces KV cache size but does not fundamentally change what the model can learn. At small model sizes the shared K/V might be expected to hurt more, but the gap is minor here.

The **latency picture tells a different story**. Local attention is the slowest by a large margin — 1373 ms/seq at T=512 versus 664 ms/seq for standard. This is an implementation artefact: our local attention uses a masked full attention matrix rather than a true sparse kernel, so it does not realise the theoretical $\mathcal{O}(T \cdot w)$ savings at the sizes we tested. An optimised implementation (e.g. using block-sparse CUDA kernels as in Longformer) would be significantly faster.

MQA is the fastest variant at both context lengths. Since it shares K and V projections across heads, there is less computation per forward pass, and the smaller KV cache means less memory pressure during the autoregressive generation loop.

Sparse block attention at T=1024 sits in the middle on all metrics — slightly worse perplexity than standard, slightly less memory, roughly similar latency. The block boundary discontinuity likely hurts, especially in a small model where there is limited capacity to compensate through depth.

4 Task 2: Positional Encodings

4.1 Variants

We compared four positional encodings, all with standard attention at T=512.

Sinusoidal (absolute) adds a fixed deterministic vector to each token embedding before the first layer. The encoding is defined for any position, but positions beyond the training length are out of distribution.

Relative (Shaw et al.) adds learned relative position biases directly to attention logits. The score between positions i and j includes a learned embedding for the relative distance $i - j$, clipped at a maximum range. This directly encodes relative distance rather than absolute position.

RoPE (rotary positional embedding) rotates query and key vectors by an angle proportional to their position before computing dot products. Because rotation matrices compose, the inner product $(R_t Q_i) \cdot (R_s K_j)$ depends only on $t - s$, giving exact relative position encoding with zero additional parameters.

ALiBi (attention with linear biases) adds no positional information to token embeddings at all. Instead it subtracts a linear penalty from attention logits proportional to distance: $\text{score}(i, j) = Q_i \cdot K_j / \sqrt{d_k} - m \cdot |i - j|$, where m is a head-specific slope. The penalty is defined for any distance, which is why ALiBi is expected to extrapolate gracefully.

4.2 In-Distribution Results

Table 2: Task 2 — Positional encoding comparison on WikiText-2 (standard attention, $d_{\text{model}} = 64$, trained and evaluated at $T = 512$). All runs on NVIDIA A100.

PE	Val PPL ↓	Peak GPU (MB)	Throughput (tok/s) ↑	Latency (ms/seq) ↓
Sinusoidal (Abs.)	107.20	10,503	264.65	483.65
ALiBi	105.26	10,510	230.96	554.20
Relative (Shaw et al.)	82.88	10,587	208.48	613.96
RoPE	81.89	10,502	172.51	741.98

4.3 Extrapolation Test

All models were trained at $T = 512$ and evaluated at $T \in \{512, 1024, 2048\}$.

Table 3: Task 2 — Extrapolation test: models trained at $T = 512$, evaluated at $T \in \{512, 1024, 2048\}$. Dashes indicate the evaluation could not be completed (see text).

PE	PPL at T=512	PPL at T=1024	PPL at T=2048
Sinusoidal (Abs.)	107.19	—	—
Relative (Shaw et al.)	82.88	—	—
RoPE	81.89	—	—
ALiBi	105.26	—	—

The T=1024 and T=2048 evaluations all failed during the inference run. The most likely cause is that relative positional encoding and RoPE raise index errors when the sequence length at evaluation exceeds the training context window, since the relative embedding table and rotation frequencies are initialised for $T \leq 512$. For sinusoidal and ALiBi this should not be the case in principle — ALiBi especially is designed for exactly this test — but the inference script applied the same validation loop uniformly and the failures were not caught per-variant. This is a gap in the results we acknowledge. A corrected re-run isolating each variant's failure mode would be needed to draw any conclusions about extrapolation.

4.4 Analysis

The in-distribution results are the clearest result in the whole study. **RoPE and Relative encoding outperform Sinusoidal and ALiBi by a substantial margin** — roughly 25 PPL points. This is a meaningful gap at this scale.

Both RoPE and Relative encoding provide the model with direct relative distance information, either through rotated key-query inner products (RoPE) or through learned per-distance bias terms (Relative). Standard sinusoidal encoding only gives the model absolute position, from which it must infer relative distances indirectly. At $T = 512$ with a 2-layer model, this indirect route is clearly less effective.

ALiBi's performance is more surprising. Despite its linear penalty mechanism, which in principle encodes relative distance, it performs comparably to sinusoidal (105 vs 107 PPL). One possible reason is that the fixed linear penalty imposes a hard locality bias — all distant tokens are penalised equally regardless of their actual relevance. For a language modelling task where long-range dependencies do sometimes matter, this constant suppression of distant attention may hurt.

RoPE's slight edge over Relative (81.89 vs 82.88) is consistent with the literature. RoPE achieves exact relative encoding with no additional learned parameters, while Relative encoding's bias terms need to be learned from data — a harder problem at small model sizes.

Memory usage is nearly identical across all four variants, which is expected since the positional encoding does not change the dimensions of the attention matrix.

5 Task 3: Conv–Attention Hybrids

5.1 Design

We added 1D convolutional components to the sparse attention + relative PE configuration, which represented a middle-ground point in the attention and PE search spaces. Two designs were tested:

Pre-attention Conv1D: a 1D convolution with kernel size 5 is applied to the token representations before each attention block. This gives the model an explicit local n-gram feature extractor that runs before the attention mechanism sees the input.

Interleaved Conv1D: Conv1D and attention blocks alternate. Rather than pairing every attention block with a convolution, some layers are purely convolutional and some are purely attention. This reduces the total number of attention computations and replaces them with cheaper convolutions.

5.2 Results

Table 4: Task 3 — Conv1D hybrid results on WikiText-2 (sparse attention + relative PE, $d_{\text{model}} = 64$). † indicates a result that could not be verified as correct (see text). Baseline row shows sparse + relative without any convolutional component.

Conv	Context	Val PPL ↓	Peak GPU (MB)	Throughput (tok/s) ↑	Latency (ms/seq) ↓
None (baseline)	512	107.83	20,241	186.00	688.16
Interleaved	512	†	10,257	467.95	273.54
Pre-Attention	1024	†	20,585	188.14	680.35

5.3 A Note on the Conv Results

The validation perplexity logged for both conv runs was approximately 1.0, which corresponds to near-zero cross-entropy loss. This is not a plausible result for a language model on WikiText-2 and almost certainly reflects a bug in how validation was run for the conv variants rather than a genuine model result. The most likely cause is that the convolutional layers introduce a data leakage path when the validation loop reuses the same batch indices, or that the causal masking was broken for the conv pathway.

We report the throughput and memory numbers, which are hardware measurements independent of the model's correctness, but mark the PPL values as unverified. Rerunning the conv experiments with a corrected validation loop is needed before any conclusions about perplexity can be drawn.

What we can say from the hardware numbers: the interleaved design at T=512 achieves notably higher throughput (467 tok/s vs 186 tok/s for the baseline) and lower latency (273 ms vs 688 ms). Replacing half the attention layers with convolutions reduces the quadratic attention compute significantly, which shows up directly in speed. The pre-attention design at T=1024 matches the baseline's throughput closely, suggesting the added convolution is not the bottleneck at that context length.

6 Discussion

6.1 What the results say

Across Tasks 1 and 2, the clearest pattern is that **inductive bias matching matters more than architectural complexity**. Local attention outperforms standard attention on WikiText-2 not because it is more powerful, but because its locality bias fits the data. RoPE and Relative outperform Sinusoidal not because they have more parameters, but because they give the model the right kind of positional information directly.

This pattern probably weakens at larger scales. With more layers, more data, and larger models, standard attention can learn to approximate local patterns, and sinusoidal encoding can be used more effectively. At the small scale of this study, explicit inductive bias is doing a lot of work.

6.2 Limitations

We list the main limitations that affect interpretation of the results:

1. **Dataset size.** WikiText-2 is too small for the models we are comparing. Overfitting forced us to use aggressive dropout and small embedding dimensions, which may compress differences between variants that would be more visible at larger scale.
2. **Hardware inconsistency.** T=512 and T=1024 runs were on different GPUs (A100 vs L40S). All latency and memory comparisons across context lengths are confounded.
3. **Extrapolation test incomplete.** The T=1024 and T=2048 evaluations failed for all PE variants. The extrapolation table cannot be used to draw any conclusions about length generalisation.
4. **Conv results unverified.** Val PPL ≈ 1.0 for both conv variants is not a valid result. These runs need to be repeated with a corrected validation setup.
5. **Sparse attention missing at T=512.** The sparse T=512 run was not completed. Cross-context-length comparisons for sparse attention are therefore not possible.
6. **Controlled comparisons only.** Task 1 fixes positional encoding to sinusoidal while varying attention. Task 2 fixes attention to standard while varying PE. The full cross-product was not run. The best combination of attention and PE might differ from what either task individually suggests.

7 Conclusion

We trained and evaluated a family of decoder-only Transformers on WikiText-2, varying attention mechanism, positional encoding, and the presence of convolutional components.

On attention variants, local attention gives the best perplexity at both T=512 and T=1024, at the cost of high latency in our unoptimised implementation. MQA matches standard attention in perplexity while being faster at inference. On positional encodings, RoPE and Relative encoding substantially outperform Sinusoidal and ALiBi in-distribution, suggesting that direct relative position information is important at small model sizes. The Conv hybrid results could not be verified due to a validation bug and require a follow-up run.

The study has real limitations, mostly driven by compute and time constraints. The cleaner takeaway is methodological: the modular codebase and experiment tracking infrastructure work well, and a larger-scale rerun on a bigger dataset with consistent hardware would give more reliable conclusions.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[2] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[3] N. Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[4] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.

[5] P. Shaw, J. Uszkoreit, and A. Vaswani. Self-attention with relative position representations. *Proceedings of NAACL-HLT*, 2018.

[6] O. Press, N. A. Smith, and M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *International Conference on Learning Representations*, 2022.